SQL Injection PYQ's

Q3. Consider the following PHP script for a login page: [20]

```
$username = $_GET[user];
$password = $_GET[pwd];
$sql = "SELECT *
        FROM usertable
        WHERE username = '$username'
        AND password = '$password' ";
$result = $db->query($sql);
if ($result->num_rows > 0) { /* Success */ }
else { /* Failure */ }
```

i. Because addslashes did not eliminate the problem of SQL injection, we decide to employ some form of input filtering. A recommends the use of JavaScript to validate the input before passing it over to the server. Is this enough to ensure that SQL injection will not happen? Why?

ii. As an extension to input filtering, we decide to strip the -- characters in the input so that attackers cannot inject a comment and thereby execute an injection. Can an attacker bypass this restriction? How?

Solve (i)

Answer (i):
No, using JavaScript for input validation is **not sufficient** to prevent SQL injection.

Reason:

- JavaScript validation happens on the **client-side**, which is **fully under the attacker's control**.
- An attacker can:
  - Disable JavaScript in the browser
  - Modify the request using tools like Postman, Burp Suite, or curl
  - Directly send malicious inputs to the server

So, any validation done in JavaScript can be **bypassed easily**.

Conclusion:
To properly prevent SQL injection, validation must be enforced on the **server-side**, and secure techniques like:

- **Prepared statements (parameterized queries)**
- **Stored procedures**
- **Proper escaping and input sanitization**

must be used.

Solve (ii)

Answer (ii):
Yes, an attacker **can still bypass this restriction**.

Reason:
Removing only the `--` comment sequence is **not enough**, because SQL provides multiple other ways to manipulate queries.

How it can be bypassed:

- Use **alternative comment styles**:
  - `#` (MySQL comment)
  - `/* ... */` (block comments)
- Avoid comments entirely and still inject logic:
  - Example: `' OR '1'='1`
- Use logical conditions to alter query behavior without comments
- Use encoding or variations to evade simple filters

Example Attack Input:

```
username: ' OR '1'='1
password: anything
```

This modifies the query to:

```
WHERE username = '' OR '1'='1' AND password = 'anything'
```

which can evaluate to **true**, allowing login bypass.

Conclusion:
Simply stripping `--` is **incomplete and insecure**. Attackers can easily use other SQL features to perform injection. Proper defense requires:

- **Prepared statements (parameterized queries)**
- **Server-side validation**
- Avoiding direct string concatenation in SQL queries.

iii. Stored Procedures have a reputation for being able to defend against SQl injection attacks. With this in mind, we decide to employ the following stored procedure:

```
CREATE PROCEDURE VerifyUser
        @username varchar(50),
        @password varchar(50)
AS
BEGIN
        SELECT * FROM UserTable
        WHERE UserName = @username
        AND Password = @password;
END
GO
```

The stored procedure is called from the PHP file using the following code:

```
$sql = "CALL VerifyUser ("+ $_GET[user] + "," + $_GET[pwd]+")";
```

Is the web application susceptible to an SQL injection? Why or why not?

Answer (iii):
Yes, the web application is **still susceptible to SQL injection**.

Reason:
Although **stored procedures** are generally considered safer, they are only secure **if used correctly with parameterized inputs**.

In this case, the procedure is called as:

```
$sql = "CALL VerifyUser ('" + $_GET[user] + "', '" + $_GET[pwd] + "')";
```

Here, user inputs are **directly concatenated into the SQL query string**, which makes it vulnerable.

Why this is dangerous:

- The attacker can inject malicious SQL inside `$_GET[user]` or `$_GET[pwd]`
- Since inputs are not bound as parameters, they are treated as part of the SQL command

Example Attack:

```
username: ' OR '1'='1
password: anything
```

This can alter the procedure call and potentially bypass authentication.

Key Insight:

- Stored procedures **do NOT automatically prevent SQL injection**
- They are safe **only when parameters are properly bound**, not when queries are built using string concatenation

Conclusion:
The application is vulnerable because:

- It uses **dynamic query construction**
- It does **not use parameterized procedure calls**

Secure Approach:

- Use **prepared statements with bound parameters** when calling the stored procedure

- Avoid concatenating user input into SQL queries

Q4. Assume that multiple groups simultaneously query the same XML to specify access policies at various levels of granularity. Suggest ways to efficiently enforce those access policies? Compare the XACML and XACL models of XML security. [15]

Answer (Q4):

Efficient Enforcement of Access Policies on XML

When multiple groups query the same XML with different access policies, efficiency can be achieved by:

1. **Fine-Grained Access Control**
   - Define policies at element/attribute level (not whole document)
   - Ensures only required parts of XML are exposed
2. **Policy Caching**
   - Cache frequently used access control decisions
   - Reduces repeated policy evaluation overhead
3. **Preprocessing / View-Based Access**
   - Create **filtered XML views** for different user groups
   - Avoids evaluating policies repeatedly for each query
4. **Indexing**
   - Use XML indexing to quickly locate nodes and enforce policies efficiently
5. **Policy Combination Strategy**
   - Use conflict resolution rules (deny-overrides, permit-overrides)
6. **Separation of Policy and Data**
   - Keep access policies separate from XML data for better scalability

---

Comparison: XACML vs XACL

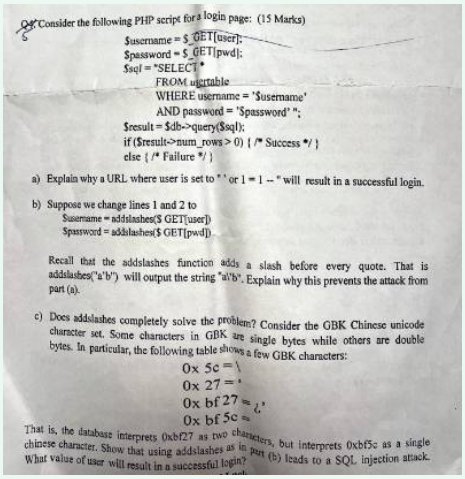
Feature	XACML (eXtensible Access Control Markup Language)	XACL (XML Access Control Language)
Type	General-purpose access control standard	XML-specific access control model
Granularity	Very fine-grained (supports complex conditions)	Fine-grained (focused on XML nodes)
Policy Expression	XML-based, highly expressive	Simpler and more specific to XML
Decision Engine	Uses PDP (Policy Decision Point) & PEP	Typically embedded with XML processing

Feature	XACML (eXtensible Access Control Markup Language)	XACL (XML Access Control Language)
Flexibility	High (supports roles, attributes, context)	Limited compared to XACML
Complexity	Complex to implement	Relatively simpler
Use Case	Enterprise-level systems, distributed environments	XML document protection
Performance	Slower due to complexity	Faster for XML-specific use cases

---

Conclusion

- **XACML** is more powerful and flexible, suitable for complex, enterprise-level access control.
- **XACL** is simpler and more efficient for XML-specific scenarios with less complexity.
- For multiple groups and dynamic policies, **XACML is generally preferred**, though **XACL can be faster for simpler use cases**.



Q8 Consider the following PHP script for a login page: (15 Marks)

```
$username = $_GET[user];
$password = $_GET[pwd];
$sql = "SELECT *
        FROM usertable
        WHERE username = '$username'
        AND password = '$password' ";
$result = $db->query($sql);
if ($result->num_rows > 0) { /* Success */ }
else { /* Failure */ }
```

a) Explain why a URL where user is set to " ' or 1 = 1 -- " will result in a successful login.

b) Suppose we change lines 1 and 2 to

```
$username = addslashes($_GET[user])
$password = addslashes($_GET[pwd])
```

Recall that the addslashes function adds a slash before every quote. That is addslashes("a'b") will output the string "a\'b". Explain why this prevents the attack from part (a).

c) Does addslashes completely solve the problem? Consider the GBK Chinese unicode character set. Some characters in GBK are single bytes while others are double bytes. In particular, the following table shows a few GBK characters:

0x 5c = \
0x 27 = '
0x bf 27 = ¿'
0x bf 5c =

That is, the database interprets 0xbf27 as two characters, but interprets 0xbf5c as a single chinese character. Show that using addslashes as in part (b) leads to a SQL injection attack. What value of user will result in a successful login?

Answer (Q7)

(a) Why `' OR 1=1 --` results in successful login

The query becomes:

```
SELECT * FROM usertable
WHERE username = '' OR 1=1 -- '
AND password = '...'
```

Explanation:

- `' OR 1=1` makes the condition always **TRUE**
- `--` comments out the rest of the query (including password check)

- So the query effectively becomes:

  ```
  WHERE TRUE
  ```

- This returns at least one row → **login success**

(b) Why `addslashes()` prevents this attack

- **`addslashes()`** converts:

  ```
  ' → \'
  ```

- So input:

  ```
  ' OR 1=1 --
  ```

  becomes:

  ```
  \' OR 1=1 --
  ```

Effect:

- The quote is **escaped**, so it is treated as part of the string
- The SQL query remains syntactically correct
- Injection fails because attacker cannot break out of the string

---

(c) Does `addslashes()` completely solve the problem? (GBK Case)

No, it does NOT completely solve the problem.

Reason (GBK Encoding Issue):

- In GBK, some characters are **2-byte sequences**
- Example:
  - **`0xbf5c`** is treated as **one character**
  - But **`0x5c`** is \ (escape character)

Attack Idea:

- Attacker crafts input so that:
  - **`addslashes()`** inserts \ (0x5c)
  - But DB interprets it as part of a **multibyte character**, not as escape

Malicious Input:

```
user = 0xbf27
```

After **`addslashes()`**:

```
0xbf5c27
```

Interpretation:

- DB reads **`0xbf5c`** as **one character**

- Remaining `'` (0x27) is now **unescaped**
- This breaks the query → allows injection

Final Answer for (c):

A possible malicious input is:

```
user = %bf%27 OR 1=1 --
```

Conclusion:

- `addslashes()` is **not secure**
- It fails under certain encodings like GBK
- Proper solution:
  - Use **prepared statements (parameterized queries)**
  - Avoid manual escaping completely

If you want, I can convert all of this into **perfect exam-format (6-mark answer style)**.

Q3. Consider the following PHP script for a login page: [15]

```
$username = $_GET[user];
$password = $_GET[pwd];
$sql = "SELECT *
        FROM usertable
        WHERE username = '$username'
        AND password = '$password' ";
$result = $db->query($sql);
if ($result->num_rows > 0) { /* Success */ }
else { /* Failure */ }
```

i. Because addslashes did not eliminate the problem of SQL injection, we decide to employ some form of input filtering. A recommends the use of JavaScript to validate the input before passing it over to the server. Is this enough to ensure that SQL injection will not happen? Why?

ii. As an extension to input filtering, we decide to strip the -- characters in the input so that attackers cannot inject a comment and thereby execute an injection. Can an attacker bypass this restriction? How?

iii. Stored Procedures have a reputation for being able to defend against SQl injection attacks. With this in mind, we decide to employ the following stored procedure:

```
CREATE PROCEDURE VerifyUser
        @username varchar(50),
        @password varchar(50)
AS
BEGIN
        SELECT * FROM UserTable
        WHERE UserName = @username
        AND Password = @password;
END
GO
```

The stored procedure is called from the PHP file using the following code:

```
$sql = "CALL VerifyUser ("+ $_GET[user] + "," + $_GET[pwd]+")";
```

3

Is the web application susceptible to an SQL injection? Why or why not?

Q3 (iii) Answer (Exam-Ready)

Yes, the web application is still susceptible to SQL injection.

Explanation:

- Although a **stored procedure (`VerifyUser`)** is used, the way it is **called from PHP is insecure**.
- The procedure call is constructed as:

  ```
  $sql = "CALL VerifyUser ('" + $_GET[user] + "', '" + $_GET[pwd] + "')";
  ```

- Here, user inputs are **directly concatenated into the SQL query string**.

Why this is vulnerable:

- The inputs are **not parameterized**
- An attacker can inject malicious SQL into **`$_GET[user]`** or **`$_GET[pwd]`**
- The database will interpret the injected input as part of the SQL command

Example Attack:

```
user = ' OR '1'='1
pwd = anything
```

This can manipulate the query logic and may result in **authentication bypass**.

Key Point:

- Stored procedures are **not inherently safe**
- They prevent SQL injection **only when used with properly bound parameters**
- Using string concatenation defeats their security benefits

Conclusion:
The application is vulnerable because it:

- Uses **dynamic SQL construction**
- Does **not use parameterized queries**

Secure Solution:

- Use **prepared statements / parameter binding** when calling the stored procedure
- Avoid direct concatenation of user inputs in SQL queries

solve (i) and (ii) also

Q3 (i) Answer (Exam-Ready)

No, using JavaScript validation is not sufficient to prevent SQL injection.

Explanation:

- JavaScript validation is performed on the **client-side**
- Attackers can easily bypass it by:
  - Disabling JavaScript
  - Modifying requests using tools (e.g., Postman, Burp Suite)
- The server still receives **untrusted input**

Conclusion:
Client-side validation alone cannot prevent SQL injection.
Server-side validation and parameterized queries are necessary.

Q3 (ii) Answer (Exam-Ready)

Yes, an attacker can bypass this restriction.

Explanation:

- Removing only `--` (SQL comment) is **not sufficient**
- Attackers can use:
  - **Alternative comments:** `#`, `/* ... */`
  - **No comments at all**, e.g.:

    ```
    ' OR '1'='1
    ```
- This alters the query logic without needing `--`

Example:

```
username = ' OR '1'='1
password = anything
```

This makes the condition **always true**, leading to login bypass.

Conclusion:
Stripping `--` alone is ineffective. SQL injection can still occur using other techniques.

Q5. a) Assume that multiple groups simultaneously query the same XML to specify access policies at various levels of granularity. Suggest ways to efficiently enforce those access policies? Compare the XACML and XACL models of XML security. [10]

b) Explain various Types of XML Injection Attacks with suitable examples? [5]

Q5 (a) Answer (Exam-Ready)

Efficient Enforcement of Access Policies on XML

To efficiently enforce access policies when multiple groups query the same XML:

1. **Fine-Grained Access Control**
   - Apply policies at **element/attribute level**
   - Ensures only required data is accessible
2. **View-Based Access Control**
   - Create **custom XML views** for different user groups
   - Reduces repeated policy evaluation
3. **Policy Caching**
   - Cache frequently used access decisions
   - Improves performance for repeated queries
4. **Indexing XML Data**
   - Use indexing to quickly locate nodes and enforce policies
5. **Policy Combination Rules**

- Use strategies like **deny-overrides / permit-overrides**

6. **Separation of Policy and Data**
   - Store policies separately for better scalability and management

---

Comparison: XACML vs XACL

Feature	XACML	XACL
Full Form	eXtensible Access Control Markup Language	XML Access Control Language
Scope	General-purpose access control	XML-specific
Granularity	Very fine-grained	Fine-grained (XML nodes)
Flexibility	High (roles, attributes, context-aware)	Limited
Complexity	Complex	Simpler
Performance	Slower (due to complexity)	Faster
Use Case	Enterprise systems	XML document security

Conclusion:

- **XACML** → more powerful & flexible
- **XACL** → simpler & efficient for XML-specific use

Q5 (b) Answer (Exam-Ready)

Types of XML Injection Attacks (with Examples)

1. **XPath Injection**
   - Similar to SQL injection but targets XPath queries
   - **Example:**

     ```
     //user[username='' or '1'='1' and password='']
     ```

   - Bypasses authentication

---

2. **XML Tag Injection**
   - Attacker injects malicious XML tags into input
   - **Example:**

     ```
     <user>Samyak</user><admin>true</admin>
     ```

   - Can escalate privileges

---

3. **XML External Entity (XXE) Attack**
   - Exploits XML parser to read local files
   - **Example:**

```
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<data>&xxe;</data>
```

- Leaks sensitive data

---

4. **Schema Injection**
   - Manipulates XML schema to bypass validation
   - **Example:**
     - Altering schema to accept unauthorized values

---

5. **Blind XPath Injection**
   - No direct output, attacker infers data via responses
   - **Example:**

     ```
     //user[username='admin' and substring(password,1,1)='a']
     ```

   - Used to extract data character by character

Conclusion

XML injections exploit improper input handling in XML processing.
Prevention methods:

- Input validation & sanitization
- Use parameterized queries for XPath
- Disable external entities (prevent XXE)
- Secure XML parsers

If you want, I can compress this into a **perfect 5-mark short answer version** for exams.